UNIDADES TECNOLÓGICAS DE SANTANDER · UTS
Tecnología en Desarrollo de Sistemas Informáticos

Sistema de Gestión de Proyectos de Grado

Asignatura
Programación JAVA

Estudiante
Erika Viviana Gualdrón Parra

Docente
Julian Barney Jaimes

Fecha
Octubre de 2026

Resumen

Este documento describe el Sistema de Gestión de Proyectos de Grado: quiénes lo usan, qué debe hacer, cómo se organiza en módulos y fases, cómo se almacena la información y cómo se organiza el código. Se elaboró con base en la entrevista con el docente.

Contenido

1. Introducción 3
1.1 Propósito del documento . . . 3
1.2 Contexto y problema . . . 3
1.3 Objetivos . . . 3
1.4 Evidencia de la reunión con el docente . . . 4
2. Descripción general del sistema 4
2.1 Roles de usuario . . . 4
2.2 Restricciones y arquitectura tecnológica . . . 5
3. Requerimientos funcionales 6
4. Requerimientos no funcionales 10
5. Módulos del sistema 11
5.1 Matriz de acceso por rol . . . 11
5.2 Menú lateral por rol . . . 11
6. Fases del trabajo de grado 12
6.1 Detalle por fase . . . 12
6.2 Estados de decisión . . . 13
6.3 Plazos, prórroga y cambio de director . . . 13
7. Diseño de la base de datos (MER) 14
7.1 Modelo entidad-relación . . . 14
7.2 Descripción de las entidades . . . 16
7.3 Relaciones entre entidades . . . 18
7.4 Diccionario de datos . . . 19
8. Historias de usuario 26
8.1 Formato y criterios . . . 26
8.2 Cuentas, usuarios y perfil . . . 26
8.3 Banco de ideas y propuestas . . . 27
8.4 Fases y seguimiento . . . 28
8.5 PQRS, historial, notificaciones e informes . . . 28
9. Estructura del proyecto: carpetas y archivos 30
9.1 Árbol de carpetas . . . 30
9.2 Descripción de carpetas . . . 31
9.3 Descripción de archivos . . . 31
9.4 Flujo típico de una página . . . 34
Anexo A. Script SQL de la base de datos (MySQL 8) 35

1. Introducción

1.1 Propósito del documento

Este documento especifica, de forma detallada y organizada, el **Sistema de Gestión de Proyectos de Grado** que será desarrollado en la asignatura Programación JAVA. Recoge los requerimientos funcionales y no funcionales, los módulos y fases del proceso, el diseño de la base de datos (modelo entidad-relación), las historias de usuario y la estructura de carpetas y archivos del proyecto, de modo que sirva como referencia única para construir, probar y entregar el sistema.

1.2 Contexto y problema

En el programa de Tecnología en Desarrollo de Sistemas Informáticos de las UTS, el trabajo de grado atraviesa varias etapas (pago, propuesta, informes y revisión final) en las que intervienen estudiantes, directores, evaluadores y el comité. Hoy ese seguimiento depende de formatos y comunicaciones dispersas, lo que dificulta saber en qué fase está cada proyecto, quién debe actuar y qué versión de cada documento es la vigente. El sistema centraliza el banco de ideas, las propuestas, el avance por fases, las solicitudes (PQRS) y el historial de cambios en una sola plataforma web.

1.3 Objetivos

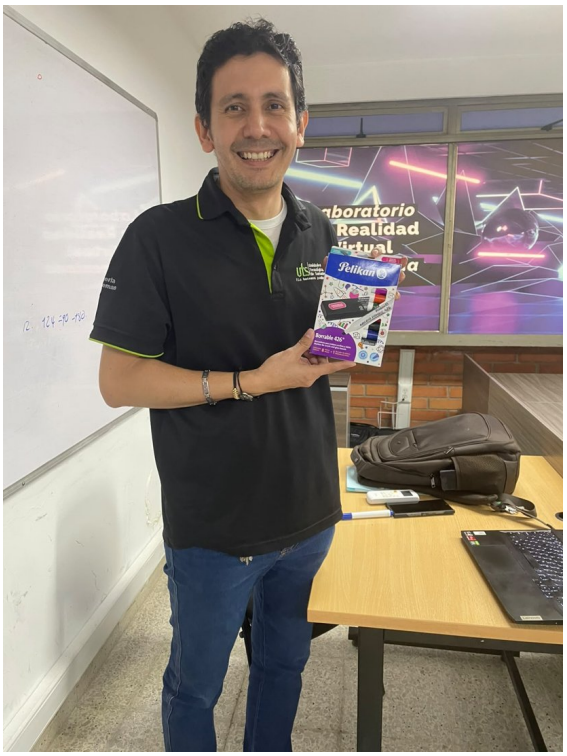
Objetivo general. Desarrollar una aplicación web que permita gestionar de principio a fin los proyectos de grado: desde la creación de la cuenta del estudiante y la selección o propuesta de una idea, hasta la aprobación final del trabajo, con control por roles, seguimiento por fases e historial de versiones.

Objetivos específicos.

- Controlar el acceso al sistema mediante cinco roles y la aprobación de cuentas de estudiantes por parte del comité.
- Administrar el banco de ideas de los docentes y las propuestas de los estudiantes con tres estados de decisión (aprobado, aplazado, rechazado).
- Dar seguimiento visual a las seis fases del trabajo de grado, con responsables, documentos y decisiones por fase.
- Registrar solicitudes de prórroga y de cambio de director a través de PQRS.
- Conservar un historial de versiones de los documentos y de los cambios de estado de cada proyecto.
- Facilitar la consulta de la base documental institucional (calendario, formatos y reglamentos).

1.4 Evidencia de la reunión con el docente

La recolección de requerimientos se realizó en una reunión presencial con el docente, en la que se mostraron las pantallas del sistema de referencia y se aclararon las dudas del documento de apoyo. Las siguientes fotografías dejan constancia de esa reunión.



Evidencia 1. Docente durante la reunión de recolección de requerimientos.



Evidencia 2. Estudiantes del equipo junto al docente al finalizar la reunión.

2. Descripción general del sistema

2.1 Roles de usuario

El sistema maneja **cinco roles**. El Administrador y el Coordinador forman el **comité**. Un docente puede actuar como Director y como Evaluador (al ingresar ve ambos módulos), con la restricción de que **nunca puede ser director y evaluador del mismo proyecto**.

Rol	Quién es / cómo se crea	Responsabilidades principales
Administrador	Creado por el sistema (semilla inicial) o por otro administrador. Integra el comité.	Aprueba o rechaza cuentas de estudiantes y asigna su nivel (Tecnológico o Profesional); registra docentes y coordinadores; consulta, edita y elimina (inactiva) usuarios; puede modificar los datos bloqueados del perfil; administra los enlaces de la base documental.

Rol	Quién es / cómo se crea	Responsabilidades principales
Coordinador	Registrado por el administrador. Integra el comité.	Revisa ideas de docentes y propuestas de estudiantes (aprueba, aplaza, rechaza; no crea ideas); asigna director y evaluador; valida la fase de pago; resuelve PQRS (prórrogas y cambios de director); consulta el proyecto de un estudiante por código o nombre; genera informes de estado.
Estudiante	Se registra solo con correo institucional; queda pendiente hasta que el comité lo apruebe.	Consulta el banco de ideas; selecciona o propone una idea (título, objetivo y alcance) y puede sumar a otro estudiante por su número de documento; entrega documentos por fase; consulta su seguimiento, historial y director/evaluador; radica PQRS.
Docente Director	Docente registrado por el administrador; asume el rol cuando el coordinador lo asigna a un proyecto.	Ve la información del estudiante; revisa y califica propuesta e informe; aprueba el envío al evaluador y al comité (Fases 3 y 5); consulta calendario y formatos.
Docente Evaluador	El mismo docente, cuando el coordinador lo asigna como evaluador.	Revisa lo enviado por el director junto con el comité y decide (aprobar o aplazar); participa en la revisión final (Fase 6). Recibe aviso cuando se cambia el director.

Además, los docentes proponen ideas para el banco de ideas desde su sesión; esas ideas quedan en Propuestas Banco hasta que el coordinador las apruebe.

2.2 Restricciones y arquitectura tecnológica

La asignatura establece las siguientes restricciones, que determinan el diseño técnico:

- **Solo JSTL, sin scriptlets:** las páginas JSP no contienen código Java incrustado (<% %>); toda la lógica de presentación y acceso a datos usa las etiquetas JSTL (core, fmt, functions y sql) y expresiones EL.
- **Sin Maven y sin archivos .java:** el proyecto es una aplicación web por carpetas, solo con JSP/JSTL, HTML, CSS y JavaScript; las dependencias (por ejemplo, JSTL y el conector MySQL) se incluyen como librerías en el proyecto.
- **Base de datos MySQL.**
- **Control de versiones con GitHub**.
- **Despliegue en la nube** en Render o Clever Cloud (contenedor Tomcat más base MySQL, según la disponibilidad del servicio elegido).

Organización lógica. Aunque no hay clases Java, se mantiene una separación por capas dentro de los JSP: (1) plantilla común (cabecera, menú lateral y pie) incluida con *jsp:include*; (2) archivo de conexión y de seguridad compartido que se incluye al inicio de cada página; (3) páginas de vista por módulo; y (4) páginas de proceso que reciben el formulario, ejecutan las sentencias SQL con parámetros (<sql:param>) y redirigen con <c:redirect>. La estructura de carpetas y archivos se detalla en la sección 9.

3. Requerimientos funcionales

Los requerimientos se agrupan por módulo. Prioridad: **A** = alta (núcleo del sistema), **M** = media, **B** = baja (deseable). La columna Rol indica quién ejecuta la función.

3.1 Autenticación y cuentas

ID	Requerimiento	Rol	Prior.
RF-01	Registrar cuenta de estudiante solicitando nombre completo, tipo y número de documento, correo institucional y contraseña.	Estudiante	A
RF-02	Aceptar únicamente correos del dominio institucional, sin verificación por correo electrónico.	Sistema	A
RF-03	Dejar la cuenta en estado Pendiente e impedir el ingreso hasta que el comité la apruebe; informar claramente si está pendiente o rechazada.	Sistema	A
RF-04	Aprobar o rechazar cuentas de estudiantes; al aprobar, asignar obligatoriamente el nivel (Tecnológico o Profesional).	Administrador	A
RF-05	Iniciar sesión con correo y contraseña, con casilla “Mantener sesión activa”, botón de envío y enlace “Crear cuenta”.	Todos	A
RF-06	Cerrar sesión y construir el menú lateral y las tarjetas según el rol del usuario.	Todos	A
RF-07	Impedir el acceso a páginas no autorizadas para el rol (redirigir al inicio o al login).	Sistema	A

3.2 Gestión de usuarios (administrador)

ID	Requerimiento	Rol	Prior.
RF-08	Registrar coordinadores, docentes y administradores con sus datos básicos y rol.	Administrador	A
RF-09	Listar usuarios con paginación, búsqueda y filtro por rol y estado.	Administrador	A
RF-10	Ver, editar e inactivar (eliminar lógicamente) usuarios, por ejemplo docentes que ya no están.	Administrador	A
RF-11	Modificar los datos bloqueados (número de documento y correo) de cualquier usuario.	Administrador	M
RF-12	Listar las solicitudes de cuenta pendientes con acceso directo a aprobar o rechazar.	Administrador	A

3.3 Mi perfil

ID	Requerimiento	Rol	Prior.
RF-13	Mostrar nombre completo, tipo de documento (lista desplegable), número de documento, correo, rol y, para estudiantes, nivel.	Todos	A
RF-14	Permitir editar nombre completo y tipo de documento; mostrar número de documento y correo bloqueados (visibles, no editables).	Todos	A
RF-15	Ver, cambiar o eliminar la foto de perfil (mediante enlace a la imagen; si no hay, mostrar imagen por defecto).	Todos	M
RF-16	Botón Guardar que valida y guarda los cambios, con mensaje de confirmación.	Todos	A

3.4 Inicio y base documental

ID	Requerimiento	Rol	Prior.
RF-17	Pantalla de inicio con tarjetas que dirigen a los módulos disponibles para el rol.	Todos	A
RF-18	Tarjetas de Calendario académico y Base documental que abren archivos externos (Dropbox/sitio de la universidad).	Todos	A
RF-19	Base documental con seis tarjetas: Calendario académico, Formatos institucionales, Presentación de propuesta, Presentación de informe final, Reglamento de estímulos y Reglamento de trabajos de grado; cada una abre su enlace externo.	Todos	A
RF-20	Administrar (crear, editar, ordenar, desactivar) los enlaces de la base documental sin tocar el código.	Administrador	B

3.5 Banco de ideas y propuestas

ID	Requerimiento	Rol	Prior.
RF-21	Proponer una idea de trabajo de grado (título, descripción, objetivo y alcance); queda en Propuestas Banco como pendiente.	Docente	A
RF-22	Revisar ideas de docentes y decidir: aprobar (pasa al Banco de ideas), aplazar o rechazar, con observación.	Coordinador	A
RF-23	Mostrar en el Banco de ideas solo las ideas aprobadas de docentes, con búsqueda y filtro (solo lectura).	Estudiante, Coord.	A
RF-24	Seleccionar una idea del banco para generar una propuesta ligada al estudiante.	Estudiante	A
RF-25	Proponer una idea propia con título, objetivo y alcance obligatorios.	Estudiante	A
RF-26	Agregar un segundo estudiante a la propuesta buscándolo por número de documento (validar que exista, esté aprobado y esté libre).	Estudiante	M
RF-27	Decidir sobre las propuestas de estudiantes con los tres estados y observación; si es aprobada, queda ligada al estudiante y se crea el proyecto.	Coordinador	A
RF-28	Si la propuesta es aplazada, permitir que el estudiante la corrija y la reenvíe.	Estudiante	A

3.6 Propuestas Banco (CRUD tipo tabla)

ID	Requerimiento	Rol	Prior.
RF-29	Tabla con código/ID, descripción, estado (aprobado, aplazado, rechazado o pendiente) y acciones, con paginación y selector de registros por página.	Todos	A
RF-30	Filtro de búsqueda por código, descripción, estado o estudiante.	Todos	A
RF-31	Dos iconos de acción: Visualizar (ojo, muestra el detalle de la solicitud) y Gestionar (aprobar, aplazar o rechazar).	Todos	A
RF-32	Docentes solo visualizan; el coordinador visualiza y gestiona; el estudiante ve las suyas.	Sistema	A
RF-33	Opción en cada fila para ver el seguimiento del proyecto (tarjetas de fases).	Todos	A

3.7 Asignación y consulta de proyectos

ID	Requerimiento	Rol	Prior.
RF-34	Asignar director a un proyecto aprobado.	Coordinador	A
RF-35	Asignar evaluador, validando que sea distinto del director.	Coordinador	A
RF-36	Consultar el proyecto o anteproyecto de un estudiante ingresando su código/documento o nombre.	Coordinador	A
RF-37	Mostrar al estudiante la información de su director y evaluador.	Estudiante	M

3.8 Fases y seguimiento

ID	Requerimiento	Rol	Prior.
RF-38	Mostrar el seguimiento con seis tarjetas de fase: verdes las superadas y la actual, grises las pendientes.	Todos	A
RF-39	Fase 1 – Pago: el estudiante registra el comprobante (mediante enlace); se crea una notificación interna al coordinador, quien valida el pago.	Estudiante, Coord.	A
RF-40	Fase 2 – Propuesta I: el estudiante envía el anteproyecto (formato institucional FDC-124) al director, quien lo revisa y califica.	Estudiante, Director	A
RF-41	Fase 3 – Propuesta II: el director aprueba y envía la propuesta al evaluador y al comité, que emiten su decisión.	Director, Evaluador, Coord.	A
RF-42	Fase 4 – Informe I: el estudiante envía el informe al director.	Estudiante, Director	A
RF-43	Fase 5 – Informe II: el director envía el informe al evaluador y al comité.	Director, Evaluador, Coord.	A
RF-44	Fase 6 – Revisión final: evaluador y comité aprueban o aplazan; al aprobar, el estudiante queda apto para graduarse.	Evaluador, Coord.	A
RF-45	Registrar la decisión de cada fase con estado y observación; el aplazamiento devuelve la fase al estudiante.	Revisores	A
RF-46	Exigir confirmación y observación obligatoria para el estado Rechazado.	Sistema	M
RF-47	Adjuntar documentos únicamente mediante el enlace (URL) al archivo; validar que el enlace sea una URL válida.	Estudiante	A
RF-48	Ventana “Detalles del proyecto” con los documentos de la fase (informe, rejilla de evaluación, Turnitin, propuesta), su código y estado, con acceso al enlace.	Todos	A

3.9 Módulos Docente Director y Docente Evaluador

ID	Requerimiento	Rol	Prior.
RF-49	Módulo Docente Director: tabla de proyectos a cargo con ID, descripción, estado (fase y estudiante) y dos acciones; tarjetas de fase que filtran la tabla.	Docente	A
RF-50	Módulo Docente Evaluador con el mismo modelo, para proyectos donde es evaluador.	Docente	A
RF-51	Un docente ve ambos módulos, pero no puede ser director y evaluador del mismo proyecto.	Sistema	A

ID	Requerimiento	Rol	Prior.
RF-52	Calificar (CRUD) el documento del estudiante: crear, editar y consultar la revisión con observaciones.	Director, Evaluador	A

3.10 PQRS

ID	Requerimiento	Rol	Prior.
RF-53	Radicar una PQRS indicando tipo (prórroga, cambio de director, petición, queja, reclamo o sugerencia), asunto y descripción.	Estudiante	A
RF-54	Para prórroga, exigir el enlace del soporte de pago; el comité aprueba o rechaza y la respuesta queda registrada en la solicitud.	Estudiante, Coord.	A
RF-55	Para cambio de director, el coordinador aprueba y asigna el nuevo director en el sistema y se notifica al evaluador.	Coordinador	A
RF-56	Consultar el estado y la respuesta de cada PQRS, con tabla, filtro y acciones de visualizar y gestionar.	Estudiante, Coord.	A

3.11 Notificaciones, informes e historial de versiones

ID	Requerimiento	Rol	Prior.
RF-57	Mostrar notificaciones internas (campana con contador y listado) ante hechos como pago registrado, nueva entrega, decisión o cambio de director. No se envían correos.	Todos	M
RF-58	Historial de versiones por proyecto: lista cronológica de versiones de documentos y cambios de estado, con fecha, usuario, acción, estado anterior/nuevo y enlace.	Todos (según permisos)	A
RF-59	Conservar todas las versiones anteriores de un documento y marcar cuál es la vigente.	Sistema	A
RF-60	Informe de estado para el coordinador, por estudiante, por director y por evaluador (proyectos, fase y estado).	Coordinador	M
RF-61	Informe de estado para el estudiante: estado de su anteproyecto y avance del proyecto completo (fases).	Estudiante	M
RF-62	Exportar el informe a CSV o imprimirlo (deseable).	Coordinador	B

Total: 62 requerimientos funcionales.

4. Requerimientos no funcionales

ID	Categoría	Requerimiento
RNF-01	Seguridad	Las contraseñas se almacenan cifradas con hash y sal (por ejemplo, función SHA2 de MySQL); nunca en texto plano.
RNF-02	Seguridad	Todas las consultas usan sentencias parametrizadas (<sql:param>) para evitar inyección SQL.
RNF-03	Seguridad	Cada página protegida incluye un archivo de seguridad que verifica la sesión y el rol antes de mostrar o procesar contenido; el menú nunca es la única barrera.
RNF-04	Seguridad	Toda salida de datos del usuario se escapa con <c:out> (o fn:escapeXml) para prevenir XSS.
RNF-05	Seguridad	Los formularios de modificación incluyen un token anti-CSRF generado por sesión y validado en el servidor.
RNF-06	Seguridad	La validación (campos obligatorios, formato de correo, dominio, URL) se hace en el servidor y, adicionalmente, en el navegador.
RNF-07	Privacidad	Los datos personales se tratan conforme a la normativa colombiana de protección de datos (Ley 1581 de 2012); solo se solicitan los datos necesarios.
RNF-08	Usabilidad	Interfaz coherente en todos los módulos: menú lateral, tarjetas, tablas con paginación y botones con significado uniforme (ojo = visualizar, lápiz = gestionar).
RNF-09	Usabilidad	Mensajes claros de éxito y error en español, y estados con colores consistentes (verde: superada/actual; gris: pendiente).
RNF-10	Usabilidad	Diseño adaptable (responsive) para uso en computador, tableta y celular.
RNF-11	Confiabilidad	Las operaciones que afectan varias tablas (por ejemplo, aprobar propuesta y crear proyecto) usan transacciones (<sql:transaction>) para no dejar datos a medias.
RNF-12	Integridad	Claves foráneas, restricciones UNIQUE y CHECK en la base de datos; borrado lógico en lugar de físico cuando hay historial.
RNF-13	Trazabilidad	Todo cambio relevante queda en el historial de versiones con usuario y fecha; el historial no se puede editar desde la interfaz.
RNF-14	Mantenibilidad	Código organizado por módulos, con plantilla compartida, nombres consistentes y sin duplicación de la conexión ni de la seguridad.
RNF-15	Mantenibilidad	Repositorio GitHub con ramas, commits descriptivos, revisión por la compañera antes de integrar y README con instrucciones de instalación.
RNF-16	Portabilidad	La aplicación corre en Apache Tomcat con JSTL y MySQL, y es desplegable en Render o Clever Cloud; la configuración de conexión se parametriza (no queda fija en el código).
RNF-17	Compatibilidad	Funciona en las versiones recientes de Chrome, Edge y Firefox.
RNF-18	Disponibilidad	Disponibilidad objetivo durante el semestre académico; copia de seguridad de la base de datos (script SQL exportado) antes de cada entrega.
RNF-19	Restricción	Sin scriptlets, sin Maven, sin archivos .java; solo JSP con JSTL, MySQL y recursos web estáticos.
RNF-20	Restricción	Sin envío de correos ni autenticación por correo; sin requerimientos de créditos; los archivos se referencian por enlace.

5. Módulos del sistema

El sistema se organiza en módulos accesibles desde el menú lateral izquierdo y desde las tarjetas de la pantalla de inicio. Cada rol ve solo los módulos que le corresponden.

5.1 Matriz de acceso por rol

Módulo	Admin.	Coord.	Estud.	Docente
Inicio	Sí	Sí	Sí	Sí
Mi perfil	Sí	Sí	Sí	Sí
Gestión de usuarios y cuentas	Sí	—	—	—
Banco de ideas	—	Ver	Ver / elegir	Ver / proponer
Propuestas Banco	Ver	Ver y gestionar	Propias	Ver
Docente Director	—	—	—	Sí
Docente Evaluador	—	—	—	Sí
Asignación y consulta de proyectos	—	Sí	—	—
Seguimiento por fases	Ver	Ver y gestionar	Propio	Según asignación
PQRS	Ver	Ver y resolver	Radicar	—
Historial de versiones	Ver	Ver	Propio	Sus proyectos
Informes de estado	—	Sí	Propio	—
Base documental	Sí (y administra enlaces)	Sí	Sí	Sí

5.2 Menú lateral por rol

Rol	Opciones del menú lateral
Estudiante	Inicio, Mi perfil, Banco de ideas, Propuesta Banco y Base documental. Desde Propuesta Banco accede al seguimiento de su proyecto, al historial de versiones, a sus PQRS y a su informe de estado.
Docente	Inicio, Mi perfil, Banco de ideas, Propuestas Banco, Docente director, Docente evaluador y Base documental.
Coordinador	Inicio, Mi perfil, Banco de ideas, Propuestas Banco (con acciones de gestión), asignación y consulta de proyectos, PQRS, informes y Base documental.
Administrador	Inicio, Mi perfil, gestión de usuarios y solicitudes de cuenta, consulta de propuestas y PQRS, historial y Base documental (con administración de sus enlaces).

6. Fases del trabajo de grado

El proceso de grado se divide en seis fases secuenciales. El estudiante no avanza a la siguiente fase hasta que la actual sea aprobada. El proceso se realiza dentro de los plazos que fija la universidad (de 90 a 180 días); el sistema no calcula fechas límite ni cuenta días.

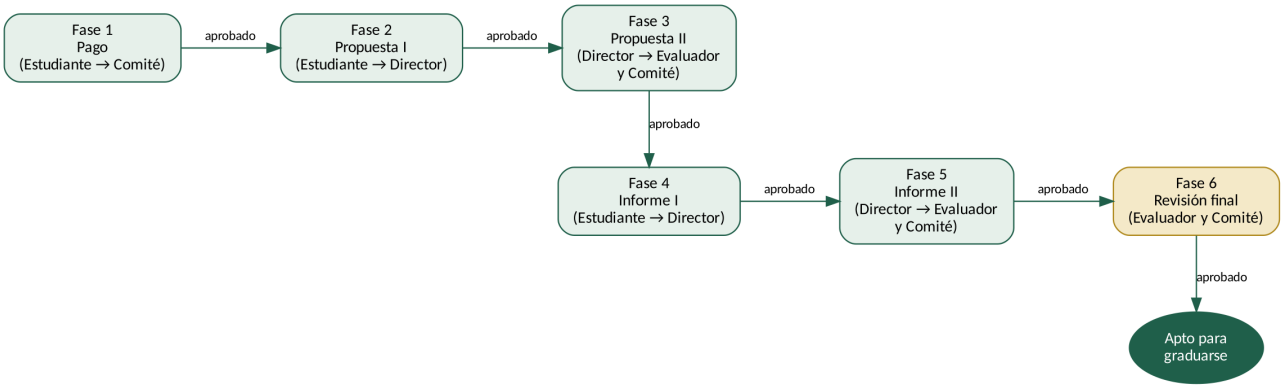


Figura 1. Flujo de las seis fases del trabajo de grado.

6.1 Detalle por fase

Fase	Qué ocurre	Quién envía / decide	Documento	Resultado
1. Pago	El estudiante sube el comprobante de pago; el sistema notifica al comité (coordinador).	Envía: estudiante. Valida: coordinador.	Comprobante de pago (enlace)	Aprobado: se habilita la Fase 2.
2. Propuesta I	El estudiante envía la propuesta (anteproyecto, formato institucional FDC-124) al director.	Envía: estudiante. Decide: director.	Propuesta/anteproyecto (enlace)	Aprobado: pasa a Fase 3. Aplazado: corrige y reenvía.
3. Propuesta II	El director envía la propuesta al evaluador y al comité.	Envía: director. Deciden: evaluador y comité.	Propuesta, rejilla de evaluación	Aprobado: pasa a Fase 4.
4. Informe I	El estudiante envía el informe de avance al director.	Envía: estudiante. Decide: director.	Informe (enlace)	Aprobado: pasa a Fase 5.
5. Informe II	El director envía el informe al evaluador y al comité.	Envía: director. Deciden: evaluador y comité.	Informe, rejilla, Turnitin	Aprobado: pasa a Fase 6.
6. Revisión final	Evaluador y comité revisan y aprueban o aplazan; es la fase final.	Deciden: evaluador y comité.	Informe final y soportes	Aprobado: el estudiante puede graduarse.

6.2 Estados de decisión

Estado	Significado	Efecto en el proceso
APROBADO	El revisor acepta el trabajo de la fase.	El proyecto avanza a la siguiente fase (o queda apto para graduarse en la Fase 6).
APLAZADO	Se devuelve con observaciones para corregir.	El estudiante permanece en la misma fase, corrige y reenvía una nueva versión; no reinicia el proceso.
RECHAZADO	Solo para casos extremos (falta gravísima).	Cierra la solicitud o el proceso; exige observación obligatoria y confirmación. Debe usarse con cuidado.
PENDIENTE	Estado técnico inicial: aún no hay decisión.	No es una decisión; se muestra mientras el revisor no responde.

6.3 Plazos, prórroga y cambio de director

- **Plazo:** el proceso completo se realiza entre 90 y 180 días según la normativa institucional. Por tratarse de un trabajo práctico, el sistema no calcula la fecha límite ni los días restantes.
- **Prórroga:** si no se alcanzó la fase final en las fechas estipuladas, el estudiante radica una PQRS de tipo prórroga y adjunta el enlace del soporte de pago; el comité la aprueba o desaprueba.
- **Cambio de director:** el estudiante lo solicita por PQRS; el coordinador realiza el cambio en el sistema y se notifica al evaluador.
- **Restricción:** el director y el evaluador de un mismo proyecto no pueden ser la misma persona.

7. Diseño de la base de datos (MER)

La base de datos se implementa en **MySQL** (motor InnoDB, codificación utf8mb4) y consta de **15 tablas**. Esta sección presenta el modelo entidad-relación completo, explica qué representa cada entidad y cada relación, y termina con el diccionario de datos de todos los campos.

7.1 Modelo entidad-relación

El diagrama de la página siguiente muestra las 15 tablas con todos sus campos, tipos, claves primarias y foráneas y las relaciones entre ellas. Se organiza de izquierda a derecha siguiendo el recorrido del proceso de grado:

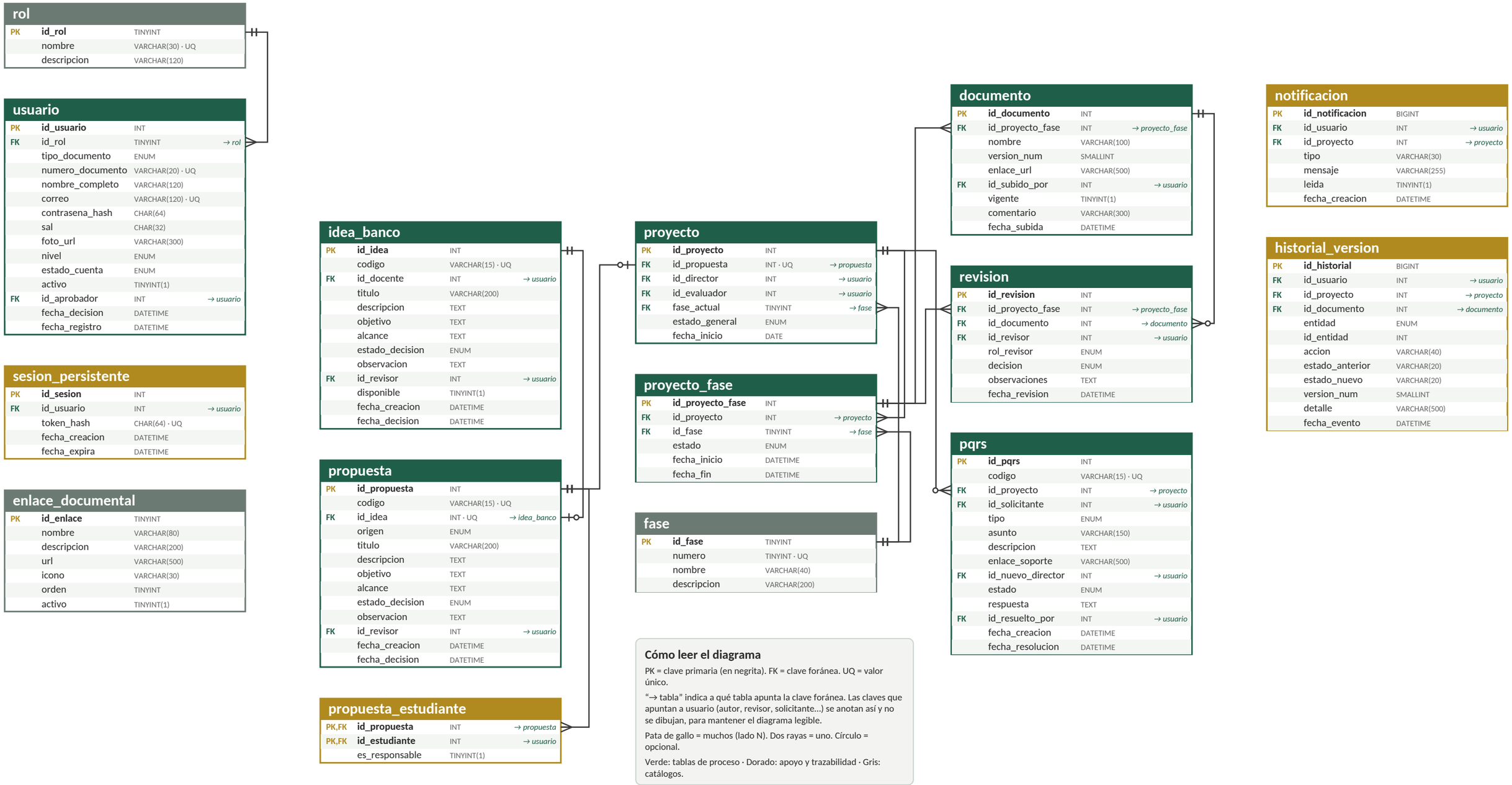
Columna del diagrama	Tablas	Qué representa
1. Seguridad y configuración	rol, usuario, sesion_persistente, enlace_documental	Quién usa el sistema, con qué rol, cómo mantiene su sesión y qué enlaces externos se muestran en la base documental.
2. Ideas y propuestas	idea_banco, propuesta, propuesta_estudiante	Las ideas que publican los docentes y las propuestas de los estudiantes (tomadas del banco o propias), con uno o dos estudiantes por propuesta.
3. Proyecto y fases	proyecto, proyecto_fase, fase	El proyecto de grado que nace de una propuesta aprobada, con su director, su evaluador y el estado de cada una de las seis fases.
4. Entregas, revisión y solicitudes	documento, revision, pqrs	Las versiones de los documentos entregados por enlace, las decisiones de los revisores y las solicitudes del estudiante (prórroga, cambio de director, PQRS).
5. Avisos y trazabilidad	notificacion, historial_version	Los avisos internos del sistema y el historial de versiones y cambios de estado.

Notación. Cada tabla muestra, por fila, la etiqueta de clave (PK o FK), el nombre del campo, su tipo en MySQL y, si es una clave foránea, la tabla a la que apunta ("→ tabla"). Las líneas del diagrama usan notación de pata de gallo:

Símbolo	Significado
Dos rayas verticales	Uno y solo uno: el registro padre debe existir.
Pata de gallo	Muchos: varios registros hijos pueden apuntar al mismo padre.
Pata de gallo con círculo	Cero o muchos: la relación es opcional del lado del hijo (la clave foránea acepta NULL).
Raya con círculo	Cero o uno: relación uno a uno opcional (la clave foránea es única, por ejemplo una idea solo puede ser tomada por una propuesta).

Nota sobre usuario. La tabla *usuario* aparece relacionada con casi todas las demás (quien propone, revisa, entrega, solicita o registra un evento). Para que el diagrama sea legible, esas claves foráneas se indican con la anotación "→ usuario" en cada tabla y no se trazan como líneas; sí se trazan todas las demás relaciones. En la sección 7.3 se listan todas, incluidas las de usuario.

Figura 2. Modelo entidad-relación completo del Sistema de Gestión de Proyectos de Grado



7.2 Descripción de las entidades

Cada entidad se explica con el papel que cumple en el sistema y los campos o reglas que más importan.

Entidad	Qué representa y para qué se usa	Campos y reglas importantes
`rol`	Catálogo de perfiles de acceso. Tiene cuatro filas: ADMINISTRADOR, COORDINADOR, ESTUDIANTE y DOCENTE. El administrador y el coordinador forman el comité. Director y Evaluador no son roles guardados: son asignaciones que el coordinador hace a un docente dentro de cada proyecto, por eso un docente puede dirigir un proyecto y evaluar otro.	nombre es único. El rol determina el menú y las páginas que el usuario puede abrir.
`usuario`	Persona que usa el sistema, sin importar su rol. Los estudiantes se registran solos y quedan pendientes; los demás los registra el administrador. Es la tabla central: casi todas las demás apuntan a ella para saber quién propuso, revisó, entregó o solicitó algo.	numero_documento y correo son únicos y están bloqueados para el usuario (solo el administrador los cambia). La contraseña se guarda como hash con sal (contrasena_hash y sal). nivel (TECNOLOGICO o PROFESIONAL) solo aplica a estudiantes y lo asigna el administrador al aprobar. estado_cuenta controla el ingreso (PENDIENTE, APROBADA, RECHAZADA); activo permite el borrado lógico. id_aprobador guarda qué administrador decidió. Una cuenta rechazada no puede tener nivel (restricción CHECK).
`sesion_persistente`	Soporta la casilla “Mantener sesión activa” del login: guarda un token (solo su hash) con fecha de expiración para reconocer al usuario en su siguiente visita.	Se borra en cascada si se elimina el usuario. token_hash es único.
`enlace_documental`	Almacena los enlaces externos de la base documental y del calendario (Dropbox o sitio de la universidad): calendario académico, formatos institucionales, presentaciones y reglamentos.	orden define la posición de la tarjeta; activo permite ocultarla sin borrarla. Así el administrador actualiza los enlaces sin tocar el código.
`idea_banco`	Idea de trabajo de grado propuesta por un docente. Queda pendiente hasta que el coordinador la revise; si la aprueba, aparece en el Banco de ideas para que los estudiantes la elijan.	estado_decision (PENDIENTE, APROBADO, APLAZADO, RECHAZADO) y observacion registran la decisión del comité; id_revisor guarda quién decidió. disponible pasa a 0 cuando un estudiante ya la tomó. codigo es único.
`propuesta`	Propuesta de trabajo de grado de un estudiante. Puede nacer de una idea del banco (origen BANCO) o ser propia (origen PROPIA, con título, objetivo y alcance). Es lo que se ve en el módulo Propuestas Banco.	id_idea es opcional y único: una idea del banco solo puede ser tomada por una propuesta. El comité decide con los tres estados; si se aplaza, el estudiante corrige y reenvía. codigo es el código visible en las tablas.
`propuesta_estudiante`	Tabla de unión entre propuesta y estudiante: resuelve que una propuesta tenga uno o dos estudiantes (el segundo se agrega por su número de documento).	La clave primaria es compuesta (id_propuesta, id_estudiante). es_responsable marca a quien creó la propuesta. Se borra en cascada con la propuesta.

Entidad	Qué representa y para qué se usa	Campos y reglas importantes
`proyecto`	Proyecto de grado. Nace cuando la propuesta es aprobada y acompaña al estudiante hasta la aprobación final. Guarda el director, el evaluador, la fase en la que va y la fecha de inicio.	id_propuesta es único (una propuesta genera un solo proyecto). id_director e id_evaluador los asigna el coordinador y deben ser distintos (restricción CHECK). estado_general indica EN_CURSO, APTO_GRADUACION o CERRADO.
`fase`	Catálogo de las seis fases: Pago, Propuesta I, Propuesta II, Informe I, Informe II y Revisión final.	numero es único. descripcion explica qué ocurre en la fase y quién actúa.
`proyecto_fase`	Estado de cada fase dentro de cada proyecto. Cada proyecto tiene seis filas, y de ahí salen las tarjetas verdes (superadas y actual) y grises (pendientes) del seguimiento.	estado: PENDIENTE, EN_CURSO, APROBADA, APLAZADA o RECHAZADA. fecha_inicio y fecha_fin registran cuándo se habilitó y cuándo se aprobó. La pareja (id_proyecto, id_fase) es única.
`documento`	Cada versión de un documento entregado por el estudiante. No guarda el archivo, sino su enlace (Drive, OneDrive, etc.).	version_num aumenta con cada reenvío tras un aplazamiento y vigente marca la versión actual; las anteriores se conservan. nombre identifica el documento (informe, rejilla, Turnitin…) y solo se guarda el enlace, sin tipo ni formato. La combinación (id_proyecto_fase, nombre, version_num) es única. id_subido_por registra quién la entregó.
`revision`	Decisión u observación que emite un revisor sobre una fase (y, opcionalmente, sobre una versión concreta del documento).	rol_revisor (DIRECTOR, EVALUADOR o COMITE) permite que en las fases 3, 5 y 6 evaluador y comité registren cada uno su decisión. decision usa los tres estados; las observaciones son obligatorias si es APLAZADO o RECHAZADO.
`pqrs`	Solicitud del estudiante: prórroga, cambio de director, petición, queja, reclamo o sugerencia. Es el canal oficial para pedir una prórroga o cambiar de director.	Para prórroga se exige enlace_soporte (soporte de pago). Para cambio de director, id_nuevo_director. estado usa los mismos estados de decisión; id_resuelto_por y respuesta registran la contestación del comité.
`notificacion`	Aviso interno del sistema (campana con contador): por ejemplo, comprobante de pago subido, nueva entrega, decisión tomada o cambio de director. No se envían correos.	leida indica si el usuario ya la vio. Se borra en cascada con el usuario. id_proyecto es opcional.
`historial_version`	Historial de versiones y cambios de estado. Cada evento relevante inserta una fila: nueva versión de documento, cambio de estado de una fase, propuesta, idea o PQRS, asignaciones y prórrogas.	Solo se inserta, no se edita desde la interfaz. Guarda usuario, fecha, entidad afectada, acción, estado anterior y nuevo, versión del documento y detalle. Un índice por (id_proyecto, fecha_evento) agiliza la consulta por proyecto.

7.3 Relaciones entre entidades

Se listan todas las relaciones del modelo, con el padre primero y el hijo después. La cardinalidad se lee de padre a hijo: “1 : N” significa que un registro del padre puede tener muchos del hijo. Las acciones de borrado no indicadas son las predeterminadas (RESTRICT): no se puede borrar un padre que tiene hijos.

Relación (padre → hijo)	Card.	Llave foránea	Explicación
rol → usuario	1 : N	`usuario.id_rol`	Cada usuario tiene exactamente un rol; un rol agrupa a muchos usuarios.
usuario → usuario	1 : N	`usuario.id_aprobador`	Autorrelación: el administrador que aprobó o rechazó la cuenta de otro usuario (opcional).
usuario → sesion_persistente	1 : N	`sesion_persistente.id_usuario`	Un usuario puede tener varias sesiones persistentes (por ejemplo, en distintos dispositivos). Borrado en cascada.
usuario (docente) → idea_banco	1 : N	`idea_banco.id_docente`	Un docente propone varias ideas; cada idea tiene un solo docente autor.
usuario (coordinador) → idea_banco	1 : N	`idea_banco.id_revisor`	El coordinador que decidió sobre la idea (opcional hasta que haya decisión).
idea_banco → propuesta	1 : 0..1	`propuesta.id_idea (único)`	Una idea puede ser tomada por, a lo sumo, una propuesta; una propuesta propia no tiene idea.
usuario (coordinador) → propuesta	1 : N	`propuesta.id_revisor`	El coordinador que decidió sobre la propuesta (opcional hasta que haya decisión).
propuesta → propuesta_estudiante	1 : 1..2	`propuesta_estudiante.id_propuesta`	Una propuesta tiene uno o dos estudiantes. Borrado en cascada.
usuario (estudiante) → propuesta_estudiante	1 : N	`propuesta_estudiante.id_estudiante`	Un estudiante participa en propuestas a través de la tabla de unión (relación N : M entre propuesta y usuario).
propuesta → proyecto	1 : 0..1	`proyecto.id_propuesta (único)`	Solo las propuestas aprobadas generan proyecto, y cada una genera uno solo.
usuario (docente) → proyecto	1 : N	`proyecto.id_director`	Un docente puede dirigir varios proyectos; cada proyecto tiene un director (asignado después de aprobar).
usuario (docente) → proyecto	1 : N	`proyecto.id_evaluador`	Un docente puede evaluar varios proyectos; el evaluador debe ser distinto del director del mismo proyecto.
fase → proyecto	1 : N	`proyecto.fase_actual`	Indica en qué fase del catálogo va cada proyecto.
proyecto → proyecto_fase	1 : 6	`proyecto_fase.id_proyecto`	Cada proyecto tiene sus seis fases, cada una con su propio estado.
fase → proyecto_fase	1 : N	`proyecto_fase.id_fase`	Cada fase del catálogo aparece en todos los proyectos.
proyecto_fase → documento	1 : N	`documento.id_proyecto_fase`	En una fase se entregan documentos y, tras cada aplazamiento, nuevas versiones.
usuario → documento	1 : N	`documento.id_subido_por`	Quién entregó cada versión.

Relación (padre → hijo)	Card.	Llave foránea	Explicación
proyecto_fase → revision	1 : N	`revision.id_proyecto_fase`	Una fase recibe varias revisiones (director, evaluador y comité).
documento → revision	1 : 0..N	`revision.id_documento`	Una revisión puede referirse a una versión concreta del documento (opcional).
usuario → revision	1 : N	`revision.id_revisor`	Quién emite cada revisión.
proyecto → pqrs	1 : 0..N	`pqrs.id_proyecto`	Un proyecto puede tener varias PQRS; la clave es opcional para PQRS generales.
usuario → pqrs	1 : N (3 veces)	`pqrs.id_solicitante`, `id_nuevo_director`, `id_resuelto_por`	Estudiante que solicita, docente propuesto como nuevo director (opcional) y coordinador que resuelve (opcional).
usuario → notificacion	1 : N	`notificacion.id_usuario`	Un usuario recibe muchas notificaciones. Borrado en cascada.
proyecto → notificacion	1 : 0..N	`notificacion.id_proyecto`	Proyecto al que se refiere el aviso (opcional).
usuario → historial_version	1 : N	`historial_version.id_usuario`	Quién realizó cada cambio registrado.
proyecto → historial_version	1 : 0..N	`historial_version.id_proyecto`	Proyecto afectado por el cambio (opcional).
documento → historial_version	1 : 0..N	`historial_version.id_documento`	Versión de documento a la que corresponde el evento (opcional).

7.4 Diccionario de datos

Para cada tabla se listan los campos, su tipo en MySQL, las restricciones y su significado. PK = clave primaria, FK = clave foránea, UNIQUE = valor irrepetible.

Tabla `rol`

Catálogo de roles de acceso al sistema.

Campo	Tipo	Restricción	Descripción
`id_rol`	`TINYINT UNSIGNED`	PK, AUTO_INCREMENT	Identificador del rol.
`nombre`	`VARCHAR(30)`	NOT NULL, UNIQUE	ADMINISTRADOR, COORDINADOR, ESTUDIANTE o DOCENTE (el docente habilita los módulos Director y Evaluador).
`descripcion`	`VARCHAR(120)`	NULL	Descripción corta del rol.

Tabla `usuario`

Todas las personas que usan el sistema, sea cual sea su rol.

Campo	Tipo	Restricción	Descripción
`id_usuario`	`INT UNSIGNED`	PK, AUTO_INCREMENT	Identificador del usuario.
`id_rol`	`TINYINT UNSIGNED`	NOT NULL, FK	Rol del usuario.
`tipo_documento`	`ENUM('CC','TI','CE','PASAPORTE','OTRO')`	NOT NULL, DEFAULT CC	Tipo de documento (lista desplegable). Editable por el usuario.

Campo	Tipo	Restricción	Descripción
numero_documento	VARCHAR(20)	NOT NULL, UNIQUE	Número de documento. Bloqueado; solo lo cambia el administrador. Sirve para agregar compañeros.
nombre_completo	VARCHAR(120)	NOT NULL	Nombre completo. Editable por el usuario.
correo	VARCHAR(120)	NOT NULL, UNIQUE	Correo institucional (dominio validado). Bloqueado.
contrasena_hash	CHAR(64)	NOT NULL	Hash SHA-256 de (sal + contraseña).
sal	CHAR(32)	NOT NULL	Sal aleatoria propia de cada usuario.
foto_url	VARCHAR(300)	NULL	Enlace a la foto de perfil; NULL = imagen por defecto.
nivel	ENUM('TECNOLOGICO','PROFESIONAL')	NULL	Nivel del estudiante, asignado por el administrador al aprobar la cuenta.
estado_cuenta	ENUM('PENDIENTE','APROBADA','RECHAZADA')	NOT NULL, DEFAULT PENDIENTE	Estado de la cuenta; solo APROBADA puede iniciar sesión.
activo	TINYINT(1)	NOT NULL, DEFAULT 1	0 = inactivo (borrado lógico).
id_aprobador	INT UNSIGNED	NULL, FK	Administrador que decidió sobre la cuenta.
fecha_decision	DATETIME	NULL	Fecha de la decisión sobre la cuenta.
fecha_registro	DATETIME	NOT NULL, DEFAULT NOW	Fecha de creación de la cuenta.

Tabla `sesion_persistente`

Tokens para la opción “Mantener sesión activa”.

Campo	Tipo	Restricción	Descripción
id_sesion	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador.
id_usuario	INT UNSIGNED	NOT NULL, FK	Dueño de la sesión.
token_hash	CHAR(64)	NOT NULL, UNIQUE	Hash del token guardado en el navegador.
fecha_creacion	DATETIME	NOT NULL, DEFAULT NOW	Cuándo se creó.
fecha_expira	DATETIME	NOT NULL	Cuándo deja de ser válido.

Tabla `idea_banco`

Ideas propuestas por docentes; al aprobarse se muestran en el Banco de ideas.

Campo	Tipo	Restricción	Descripción
id_idea	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador de la idea.
codigo	VARCHAR(15)	NOT NULL, UNIQUE	Código visible (por ejemplo, IDE-00001).
id_docente	INT UNSIGNED	NOT NULL, FK	Docente que propone la idea.
titulo	VARCHAR(200)	NOT NULL	Título de la idea.
descripcion	TEXT	NOT NULL	Descripción general (se muestra en las tablas).

Campo	Tipo	Restricción	Descripción
objetivo	TEXT	NULL	Objetivo de la idea.
alcance	TEXT	NULL	Alcance de la idea.
estado_decision	ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO')	NOT NULL, DEFAULT PENDIENTE	Decisión del coordinador.
observacion	TEXT	NULL	Observación del coordinador.
id_revisor	INT UNSIGNED	NULL, FK	Coordinador que decidió.
disponible	TINYINT(1)	NOT NULL, DEFAULT 1	0 cuando un estudiante ya la tomó.
fecha_creacion	DATETIME	NOT NULL, DEFAULT NOW	Fecha de creación.
fecha_decision	DATETIME	NULL	Fecha de la decisión.

Tabla `propuesta`

Propuesta de trabajo de grado de un estudiante (tomada del banco o propia).

Campo	Tipo	Restricción	Descripción
id_propuesta	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador de la propuesta.
codigo	VARCHAR(15)	NOT NULL, UNIQUE	Código específico visible (por ejemplo, GRA-00001).
id_idea	INT UNSIGNED	NULL, UNIQUE, FK	Idea del banco elegida; NULL si es propia.
origen	ENUM('BANCO','PROPIA')	NOT NULL	Cómo nació la propuesta.
titulo	VARCHAR(200)	NOT NULL	Título.
descripcion	TEXT	NOT NULL	Descripción mostrada en las tablas.
objetivo	TEXT	NOT NULL	Objetivo.
alcance	TEXT	NOT NULL	Alcance.
estado_decision	ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO')	NOT NULL, DEFAULT PENDIENTE	Decisión del comité.
observacion	TEXT	NULL	Observación del comité.
id_revisor	INT UNSIGNED	NULL, FK	Coordinador que decidió.
fecha_creacion	DATETIME	NOT NULL, DEFAULT NOW	Fecha de radicación.
fecha_decision	DATETIME	NULL	Fecha de la decisión.

Tabla `propuesta_estudiante`

Estudiantes de cada propuesta (uno o dos).

Campo	Tipo	Restricción	Descripción
id_propuesta	INT UNSIGNED	PK, FK	Propuesta.
id_estudiante	INT UNSIGNED	PK, FK	Estudiante integrante.

Campo	Tipo	Restricción	Descripción
es_responsable	TINYINT(1)	NOT NULL, DEFAULT 0	1 = quien creó la propuesta; 0 = compañero agregado.

Tabla fase

Catálogo de las seis fases.

Campo	Tipo	Restricción	Descripción
id_fase	TINYINT UNSIGNED	PK	Identificador (igual al número de fase).
numero	TINYINT UNSIGNED	NOT NULL, UNIQUE	Número de la fase (1 a 6).
nombre	VARCHAR(40)	NOT NULL	Pago, Propuesta I, Propuesta II, Informe I, Informe II, Revisión final.
descripcion	VARCHAR(200)	NULL	Qué ocurre en la fase y quién actúa.

Tabla proyecto

Proyecto de grado, creado cuando se aprueba la propuesta.

Campo	Tipo	Restricción	Descripción
id_proyecto	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del proyecto.
id_propuesta	INT UNSIGNED	NOT NULL, UNIQUE, FK	Propuesta aprobada de origen.
id_director	INT UNSIGNED	NULL, FK	Docente director (asignado por el coordinador).
id_evaluador	INT UNSIGNED	NULL, FK	Docente evaluador (asignado por el coordinador).
fase_actual	TINYINT UNSIGNED	NOT NULL, DEFAULT 1, FK	Fase en la que está el proyecto.
estado_general	ENUM('EN_CURSO','APTO_GRADUACION','CERRADO')	NOT NULL, DEFAULT EN_CURSO	Estado global.
fecha_inicio	DATE	NOT NULL	Fecha en que inicia el proyecto.

Tabla proyecto_fase

Estado de cada fase dentro de cada proyecto (alimenta las tarjetas verdes y grises).

Campo	Tipo	Restricción	Descripción
id_proyecto_fase	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador.
id_proyecto	INT UNSIGNED	NOT NULL, FK	Proyecto.
id_fase	TINYINT UNSIGNED	NOT NULL, FK	Fase.
estado	ENUM('PENDIENTE','EN_CURSO','APROBADA','APLAZADA','RECHAZADA')	NOT NULL, DEFAULT PENDIENTE	Estado de la fase en este proyecto.
fecha_inicio	DATETIME	NULL	Cuándo se habilitó.
fecha_fin	DATETIME	NULL	Cuándo se aprobó.

Tabla `documento`

Cada versión de un documento entregado (se guarda su enlace).

Campo	Tipo	Restricción	Descripción
`id_documento`	`INT UNSIGNED`	PK, AUTO_INCREMENT	Identificador de la versión.
`id_proyecto_fase`	`INT UNSIGNED`	NOT NULL, FK	Fase del proyecto a la que pertenece.
`nombre`	`VARCHAR(100)`	NOT NULL	Nombre del documento (por ejemplo, Informe, Rejilla de evaluación, Turnitin).
`version_num`	`SMALLINT UNSIGNED`	NOT NULL, DEFAULT 1	Número de versión (1, 2, 3…).
`enlace_url`	`VARCHAR(500)`	NOT NULL	URL del archivo (Drive, OneDrive, etc.).
`id_subido_por`	`INT UNSIGNED`	NOT NULL, FK	Quién entregó esta versión.
`vigente`	`TINYINT(1)`	NOT NULL, DEFAULT 1	1 = versión vigente; las anteriores quedan en 0.
`comentario`	`VARCHAR(300)`	NULL	Nota del estudiante sobre qué cambió.
`fecha_subida`	`DATETIME`	NOT NULL, DEFAULT NOW	Fecha de la entrega.

Tabla `revision`

Decisión u observación de un revisor sobre una fase o documento.

Campo	Tipo	Restricción	Descripción
`id_revision`	`INT UNSIGNED`	PK, AUTO_INCREMENT	Identificador.
`id_proyecto_fase`	`INT UNSIGNED`	NOT NULL, FK	Fase revisada.
`id_documento`	`INT UNSIGNED`	NULL, FK	Versión del documento revisada.
`id_revisor`	`INT UNSIGNED`	NOT NULL, FK	Quién revisa.
`rol_revisor`	`ENUM('DIRECTOR','EVALUADOR','COMITE')`	NOT NULL	Rol con el que revisa.
`decision`	`ENUM('APROBADO','APLAZADO','RECHAZADO')`	NOT NULL	Decisión.
`observaciones`	`TEXT`	NULL	Observaciones (obligatorias si es APLAZADO o RECHAZADO).
`fecha_revision`	`DATETIME`	NOT NULL, DEFAULT NOW	Fecha.

Tabla `pqrs`

Solicitudes del estudiante: prórroga, cambio de director y PQRS generales.

Campo	Tipo	Restricción	Descripción
`id_pqrs`	`INT UNSIGNED`	PK, AUTO_INCREMENT	Identificador.
`codigo`	`VARCHAR(15)`	NOT NULL, UNIQUE	Código visible (por ejemplo, PQ-00001).
`id_proyecto`	`INT UNSIGNED`	NULL, FK	Proyecto relacionado.
`id_solicitante`	`INT UNSIGNED`	NOT NULL, FK	Estudiante que radica.

Campo	Tipo	Restricción	Descripción
tipo	ENUM('PRORROGA','CAMBIO_DIRECTOR','PETICION','QUEJA','RECLAMO','SUGERENCIA')	NOT NULL	Tipo de solicitud.
asunto	VARCHAR(150)	NOT NULL	Asunto.
descripcion	TEXT	NOT NULL	Detalle.
enlace_soporte	VARCHAR(500)	NULL	Enlace al soporte (pago de la prórroga).
id_nuevo_director	INT UNSIGNED	NULL, FK	Nuevo director (cambio aprobado).
estado	ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO')	NOT NULL, DEFAULT PENDIENTE	Estado de la solicitud.
respuesta	TEXT	NULL	Respuesta del comité.
id_resuelto_por	INT UNSIGNED	NULL, FK	Quién resolvió.
fecha_creacion	DATETIME	NOT NULL, DEFAULT NOW	Fecha de radicación.
fecha_resolucion	DATETIME	NULL	Fecha de respuesta.

Tabla `notificacion`

Avisos internos del sistema (no se envían por correo).

Campo	Tipo	Restricción	Descripción
id_notificacion	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador.
id_usuario	INT UNSIGNED	NOT NULL, FK	Destinatario.
id_proyecto	INT UNSIGNED	NULL, FK	Proyecto relacionado.
tipo	VARCHAR(30)	NOT NULL	PAGO, ENTREGA, DECISION, CAMBIO_DIRECTOR, etc.
mensaje	VARCHAR(255)	NOT NULL	Texto del aviso.
leida	TINYINT(1)	NOT NULL, DEFAULT 0	0 = sin leer.
fecha_creacion	DATETIME	NOT NULL, DEFAULT NOW	Fecha del aviso.

Tabla `historial_version`

Historial de versiones y cambios de estado (solo se inserta, no se edita).

Campo	Tipo	Restricción	Descripción
id_historial	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador.
id_usuario	INT UNSIGNED	NOT NULL, FK	Quién realizó el cambio.
id_proyecto	INT UNSIGNED	NULL, FK	Proyecto afectado.
id_documento	INT UNSIGNED	NULL, FK	Documento afectado.

Campo	Tipo	Restricción	Descripción
entidad	ENUM('USUARIO','IDEA','PROPUESTA','PROYECTO','FASE','DOCUMENTO','PQRS')	NOT NULL	Tipo de registro modificado.
id_entidad	INT UNSIGNED	NOT NULL	Id del registro modificado.
accion	VARCHAR(40)	NOT NULL	CREA, NUEVA_VERSION, CAMBIO_ESTADO, ASIGNA_DIRECTOR, PRORROGA…
estado_anterior	VARCHAR(20)	NULL	Estado antes del cambio.
estado_nuevo	VARCHAR(20)	NULL	Estado después del cambio.
version_num	SMALLINT UNSIGNED	NULL	Versión del documento (si aplica).
detalle	VARCHAR(500)	NULL	Observación o descripción del cambio.
fecha_evento	DATETIME	NOT NULL, DEFAULT NOW	Fecha y hora.

Tabla `enlace_documental`

Enlaces externos de la Base documental y del calendario.

Campo	Tipo	Restricción	Descripción
id_enlace	TINYINT UNSIGNED	PK, AUTO_INCREMENT	Identificador.
nombre	VARCHAR(80)	NOT NULL	Título de la tarjeta.
descripcion	VARCHAR(200)	NULL	Texto de la tarjeta.
url	VARCHAR(500)	NOT NULL	Enlace externo (Dropbox o sitio de la universidad).
icono	VARCHAR(30)	NULL	Icono de la tarjeta.
orden	TINYINT UNSIGNED	NOT NULL, DEFAULT 0	Posición en pantalla.
activo	TINYINT(1)	NOT NULL, DEFAULT 1	1 = visible.

8. Historias de usuario

8.1 Formato y criterios

Cada historia sigue el formato **“Como [rol], quiero [funcionalidad], para [beneficio]”** e incluye criterios de aceptación, prioridad (A alta, M media, B baja) y puntos de esfuerzo (escala de Fibonacci: 1, 2, 3, 5, 8). Las historias técnicas (TE) preparan la base del proyecto.

8.2 Cuentas, usuarios y perfil

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
TE-01	**Como** Equipo, **quiero** crear el repositorio en GitHub con ramas, README y estructura de carpetas, **para** trabajar ordenadamente en pareja.	Repositorio con main y develop; README; carpetas definidas; .gitignore; tablero de proyecto creado.	A	3
TE-02	**Como** Equipo, **quiero** crear la base de datos MySQL con sus tablas y datos iniciales, **para** tener la base para todos los módulos.	Script SQL ejecuta sin errores; 15 tablas; roles, fases y enlaces cargados; administrador inicial.	A	5
TE-03	**Como** Equipo, **quiero** una plantilla común (cabecera, menú, pie), conexión y seguridad reutilizables, **para** no repetir código y proteger las páginas.	Cada página incluye conexión y seguridad; el menú cambia según el rol; sin scriptlets.	A	5
HU-01	**Como** estudiante, **quiero** crear mi cuenta con nombre completo, documento y correo institucional, **para** solicitar acceso al sistema.	Solo acepta el dominio institucional; documento y correo no repetidos; la cuenta queda Pendiente; muestra mensaje de espera.	A	5
HU-02	**Como** usuario, **quiero** iniciar sesión con correo y contraseña y mantener la sesión activa, **para** acceder a mis módulos.	Valida credenciales; bloquea cuentas pendientes, rechazadas o inactivas con mensaje claro; incluye casilla de mantener sesión y enlace a crear cuenta.	A	3
HU-03	**Como** administrador, **quiero** aprobar o rechazar cuentas de estudiantes y asignar su nivel, **para** controlar quién entra al sistema.	Lista de pendientes; al aprobar exige nivel Tecnológico o Profesional; al rechazar no asigna nivel; queda en el historial.	A	5
HU-04	**Como** administrador, **quiero** registrar docentes y coordinadores, **para** que tengan acceso sin crear cuenta propia.	Formulario con rol; correo institucional; contraseña inicial con hash; cuenta aprobada.	A	3
HU-05	**Como** administrador, **quiero** consultar, editar e inactivar usuarios, incluidos los datos bloqueados, **para** mantener la información al día.	Tabla con búsqueda y filtros; edita documento y correo; inactivar conserva el historial.	A	5
HU-06	**Como** usuario, **quiero** ver y editar mi perfil, **para** mantener mis datos correctos.	Edita nombre y tipo de documento; documento y correo visibles y bloqueados; botón Guardar con confirmación.	A	3
HU-07	**Como** usuario, **quiero** ver, cambiar o eliminar mi foto de perfil, **para** personalizar mi cuenta.	Acepta enlace de imagen; si se elimina muestra imagen por defecto.	M	2
HU-08	**Como** usuario, **quiero** ver una pantalla de inicio con tarjetas de mis módulos, **para** llegar rápido a lo que necesito.	Las tarjetas dependen del rol; las de calendario y base documental abren enlaces externos.	A	3

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
HU-09	**Como** usuario, **quiero** consultar la base documental, **para** acceder a formatos y reglamentos.	Seis tarjetas; cada una abre su enlace en otra pestaña.	M	2
HU-10	**Como** administrador, **quiero** administrar los enlaces de la base documental, **para** mantener la información vigente.	Crear, editar, ordenar y desactivar enlaces.	B	2

8.3 Banco de ideas y propuestas

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
HU-11	**Como** docente, **quiero** proponer una idea de trabajo de grado, **para** aportar temas al banco.	Título, descripción, objetivo y alcance obligatorios; queda pendiente en Propuestas Banco.	A	3
HU-12	**Como** coordinador, **quiero** revisar ideas de docentes y aprobarlas, aplazarlas o rechazarlas, **para** decidir cuáles pasan al banco.	Tres estados; observación obligatoria al aplazar o rechazar; al aprobar aparece en el Banco de ideas.	A	5
HU-13	**Como** estudiante, **quiero** ver el banco de ideas de los docentes, **para** elegir un tema.	Muestra solo ideas aprobadas y disponibles; búsqueda y paginación; solo lectura.	A	3
HU-14	**Como** estudiante, **quiero** seleccionar una idea del banco, **para** iniciar mi propuesta.	Crea una propuesta ligada a mí; marca la idea como no disponible; queda pendiente del comité.	A	3
HU-15	**Como** estudiante, **quiero** proponer mi propia idea con título, objetivo y alcance, **para** trabajar un tema de mi interés.	Campos obligatorios; se registra en Propuestas Banco ligada a mí; estado pendiente.	A	5
HU-16	**Como** estudiante, **quiero** agregar a otro estudiante por su número de documento, **para** hacer el trabajo en pareja.	Busca por documento; valida que exista, esté aprobado y libre; máximo un compañero.	M	3
HU-17	**Como** coordinador, **quiero** aprobar, aplazar o rechazar propuestas de estudiantes, **para** dar paso al proyecto.	Tres estados con observación; al aprobar se crea el proyecto en Fase 1 y se registra en el historial.	A	5
HU-18	**Como** estudiante, **quiero** corregir y reenviar una propuesta aplazada, **para** continuar sin empezar de nuevo.	Edita la propuesta aplazada; vuelve a Pendiente; el historial conserva la versión previa.	A	3
HU-19	**Como** usuario, **quiero** ver Propuestas Banco en una tabla con filtro, paginación y acciones, **para** consultar el estado de las propuestas.	Columnas ID, descripción, estado y acciones; ojo para ver detalle; lápiz solo para el coordinador; docentes solo visualizan.	A	5
HU-20	**Como** coordinador, **quiero** asignar director y evaluador a un proyecto, **para** habilitar la revisión.	Lista solo docentes activos; rechaza que director y evaluador sean la misma persona; queda en el historial.	A	5
HU-21	**Como** coordinador, **quiero** buscar a un estudiante por código o nombre, **para** ver los datos de su proyecto.	Resultado con proyecto, fase, director y evaluador.	M	3

8.4 Fases y seguimiento

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
HU-22	**Como** usuario, **quiero** ver el seguimiento del proyecto con tarjetas de fase, **para** saber en qué fase está.	Seis tarjetas: verdes las superadas y la actual, grises las pendientes; botón ver en cada una.	A	5
HU-23	**Como** estudiante, **quiero** subir el comprobante de pago (Fase 1), **para** iniciar el proceso de grado.	Enlace obligatorio; se crea notificación interna al comité; la fase queda en revisión.	A	3
HU-24	**Como** coordinador, **quiero** validar el pago, **para** habilitar la propuesta.	Aprueba o aplaza con observación; al aprobar se habilita la Fase 2.	A	3
HU-25	**Como** estudiante, **quiero** entregar propuesta o informe mediante un enlace, **para** cumplir cada fase.	Valida que sea una URL; crea nueva versión; aviso al revisor.	A	5
HU-26	**Como** director, **quiero** revisar y decidir las fases 2 y 4, **para** dar visto bueno o devolver.	Ve los documentos; registra decisión y observaciones; aplazado devuelve la fase al estudiante.	A	5
HU-27	**Como** director, **quiero** aprobar el envío de propuesta e informe al evaluador y al comité (fases 3 y 5), **para** que continúe la revisión.	Habilita al evaluador y al comité; no aprobado deja en la misma fase con observación.	A	5
HU-29	**Como** usuario, **quiero** abrir “Detalles del proyecto” con los documentos de la fase, **para** revisar informe, rejilla, Turnitin y propuesta.	Muestra cada documento con código, estado y enlace; muestra versiones anteriores.	A	3
HU-28	**Como** evaluador y comité, **quiero** decidir sobre las fases 3, 5 y 6, **para** cerrar la revisión de cada etapa.	Cada uno registra su decisión; la fase avanza cuando ambos aprueban; rechazo con confirmación y observación.	A	8
HU-30	**Como** docente, **quiero** usar el módulo Docente Director, **para** gestionar mis proyectos como director.	Tabla ID, descripción, estado (fase + estudiante) y dos acciones; tarjetas de fase filtran la tabla.	A	5
HU-31	**Como** docente, **quiero** usar el módulo Docente Evaluador, **para** gestionar mis proyectos como evaluador.	Mismo modelo; nunca aparece un proyecto donde también soy director.	A	3
HU-32	**Como** estudiante, **quiero** quedar habilitado para graduarme al aprobar la fase final, **para** cerrar mi trabajo de grado.	Fase 6 aprobada cambia el proyecto a Apto para graduación; queda en el historial.	A	2
HU-32	**Como** estudiante, **quiero** quedar habilitado para graduarme al aprobar la fase final, **para** cerrar mi trabajo de grado.	Fase 6 aprobada cambia el proyecto a Apto para graduación; queda en el historial.	A	2

8.5 PQRS, historial, notificaciones e informes

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
HU-33	**Como** estudiante, **quiero** radicar una PQRS (prórroga, cambio de director u otra), **para** pedir ajustes al proceso.	Tipo, asunto y descripción; la prórroga exige enlace del soporte de pago; consulta su estado.	A	5
HU-34	**Como** coordinador, **quiero** resolver PQRS, **para** dar respuesta a las solicitudes.	Aprueba, aplaza o rechaza con respuesta; la respuesta queda registrada en la PQRS.	A	5

ID	Historia de usuario	Criterios de aceptación	Pr.	Pts
HU-35	**Como** coordinador, **quiero** cambiar el director de un proyecto desde una PQRS aprobada, **para** atender la solicitud del estudiante.	Actualiza el director; notifica al evaluador; guarda el cambio en el historial.	A	3
HU-36	**Como** usuario, **quiero** consultar el historial de versiones del proyecto, **para** conocer qué cambió, quién y cuándo.	Lista cronológica con fecha, usuario, acción, estado anterior/nuevo, versión y enlace; solo lectura.	A	5
HU-37	**Como** usuario, **quiero** recibir notificaciones dentro del sistema, **para** enterarme de lo que requiere mi acción.	Campana con contador; listado con marcar como leída; sin correos.	M	5
HU-38	**Como** coordinador, **quiero** obtener informes de estado por estudiante, director y evaluador, **para** hacer seguimiento general.	Tabla con proyecto, fase, estado y último movimiento; filtros; exportación a CSV deseable.	M	5
HU-39	**Como** estudiante, **quiero** ver el informe de estado de mi proyecto, **para** conocer mi avance.	Estado del anteproyecto y avance de las seis fases.	M	3
HU-40	**Como** Equipo, **quiero** probar el sistema y desplegarlo en la nube, **para** entregar una versión funcionando.	Pruebas por rol sin errores críticos; despliegue en Render o Clever Cloud con MySQL; manual breve de uso.	A	5

9. Estructura del proyecto: carpetas y archivos

Esta sección describe cómo se organiza el código del sistema. Como el proyecto se desarrolla solo con JSP y JSTL (sin scriptlets, sin Maven y sin clases .java), la estructura es una aplicación web por carpetas que se despliega directamente en Apache Tomcat. Se aplican tres ideas: (1) los fragmentos comunes (conexión, seguridad, menú, cabecera y pie) se escriben una sola vez y se incluyen en las páginas; (2) cada módulo tiene su propia carpeta con sus páginas de vista; y (3) las páginas que reciben formularios y modifican datos están separadas en la carpeta *proceso*.

9.1 Árbol de carpetas

```
gestion-proyectos-grado/
├── README.md                        instrucciones de instalación y uso
├── .gitignore
├── docs/                            este documento y diagramas
├── sql/
│   ├── schema.sql                   creación de tablas (Anexo A)
│   └── datos_iniciales.sql          roles, fases y enlaces
└── webapp/                          raíz de la aplicación web
    ├── index.jsp                    inicio de sesión
    ├── registro.jsp                 crear cuenta de estudiante
    ├── inicio.jsp                   pantalla de inicio con tarjetas
    ├── cerrar_sesion.jsp
    ├── acceso_denegado.jsp
    ├── WEB-INF/
    │   ├── web.xml                  configuración de la aplicación y errores
    │   ├── lib/                     jstl y conector de MySQL (.jar)
    │   └── includes/
    │       ├── conexion.jsp         fuente de datos MySQL
    │       ├── seguridad.jsp        validación de sesión, rol y token CSRF
    │       ├── cabecera.jsp         encabezado y campana de notificaciones
    │       ├── menu.jsp             menú lateral según el rol
    │       ├── pie.jsp              cierre de la página y scripts
    │       └── paginacion.jsp       controles de paginación reutilizables
    ├── modulos/
    │   ├── perfil/                  perfil.jsp
    │   ├── usuarios/                usuarios.jsp  usuario_form.jsp  cuentas_pendientes.jsp
    │   ├── banco_ideas/             banco_ideas.jsp  idea_form.jsp
    │   ├── propuestas/              propuestas_banco.jsp  propuesta_form.jsp
    │   │                            propuesta_detalle.jsp  propuesta_decision.jsp
    │   ├── proyectos/               asignacion.jsp  buscar_estudiante.jsp
    │   │                            seguimiento.jsp  detalles_proyecto.jsp  entrega_form.jsp
    │   ├── docente_director/        docente_director.jsp
    │   ├── docente_evaluador/       docente_evaluador.jsp
    │   ├── pqrs/                    pqrs.jsp  pqrs_form.jsp  pqrs_detalle.jsp
    │   ├── historial/               historial.jsp
    │   ├── informes/                informe_coordinador.jsp  informe_estudiante.jsp
    │   ├── base_documental/         base_documental.jsp  enlaces_admin.jsp
    │   └── notificaciones/          notificaciones.jsp
    ├── proceso/                     páginas que procesan formularios
    │   ├── login_proc.jsp  registro_proc.jsp  perfil_proc.jsp  usuario_proc.jsp
    │   ├── cuenta_decision_proc.jsp  idea_proc.jsp  propuesta_proc.jsp
    │   ├── asignacion_proc.jsp  entrega_proc.jsp  revision_proc.jsp
    │   └── pqrs_proc.jsp  enlace_proc.jsp  notificacion_proc.jsp
    └── assets/
        ├── css/   estilos.css
        ├── js/    validaciones.js  tablas.js
        └── img/   logo, iconos y foto por defecto
```

9.2 Descripción de carpetas

Carpeta	Para qué sirve
docs/	Documentación del proyecto: este documento, el diagrama MER y los manuales.
sql/	Scripts de la base de datos. `schema.sql` crea las 15 tablas y sus restricciones; `datos_iniciales.sql` carga los datos que el sistema necesita para funcionar (roles, fases, tipos de documento, enlaces de la base documental y el administrador inicial).
webapp/	Raíz de la aplicación web. Todo lo que está aquí es lo que se despliega en Tomcat (Render o Clever Cloud). Contiene las páginas públicas (login y registro) y las carpetas del resto del sistema.
webapp/WEB-INF/	Carpeta protegida: el navegador no puede abrir directamente nada de lo que hay dentro. Por eso aquí van la configuración, las librerías y los fragmentos compartidos con la conexión y la seguridad.
webapp/WEB-INF/lib/	Librerías (.jar) necesarias al no usar Maven: JSTL (core, fmt, functions y sql) y el conector JDBC de MySQL.
webapp/WEB-INF/includes/	Fragmentos JSP reutilizables que se incluyen al inicio y al final de cada página (conexión, seguridad, cabecera, menú, pie y paginación).
webapp/modulos/	Una subcarpeta por módulo del sistema, con sus páginas de vista (listados, formularios y detalles). Los nombres de las carpetas coinciden con los módulos de la sección 5.
webapp/proceso/	Páginas sin interfaz que reciben los datos de un formulario, validan, ejecutan las sentencias SQL con parámetros y redirigen de vuelta a la vista con un mensaje. Separarlas evita que se reenvíe un formulario al recargar la página.
webapp/assets/	Recursos estáticos: hojas de estilo, JavaScript del navegador e imágenes.

9.3 Descripción de archivos

Raíz de la aplicación y configuración

Archivo	Función
`index.jsp`	Pantalla de inicio de sesión: correo, contraseña, casilla “Mantener sesión activa”, botón de envío y enlace a “Crear cuenta”.
`registro.jsp`	Formulario de creación de cuenta para estudiantes: nombre completo, tipo y número de documento, correo institucional y contraseña.
`inicio.jsp`	Pantalla de bienvenida con las tarjetas de acceso a los módulos según el rol y a los enlaces externos (base documental y calendario).
`cerrar_sesion.jsp`	Invalida la sesión y regresa al login.
`acceso_denegado.jsp`	Mensaje que se muestra cuando un usuario intenta abrir una página que no corresponde a su rol.
`WEB-INF/web.xml`	Configuración de la aplicación: página de inicio y páginas de error.

Fragmentos compartidos (WEB-INF/includes)

Archivo	Función
`conexion.jsp`	Define la fuente de datos MySQL con <sql:setDataSource>; es el único lugar donde se configura la conexión.
`seguridad.jsp`	Se incluye al inicio de cada página protegida: verifica que haya sesión, que el rol tenga permiso para la página y que el token anti-CSRF sea válido; si no, redirige.
`cabecera.jsp`	Encabezado común: logo, nombre y foto del usuario y campana de notificaciones con su contador.
`menu.jsp`	Menú lateral izquierdo; muestra solo las opciones del rol (sección 5.2).
`pie.jsp`	Cierre de la página y carga de los scripts.
`paginacion.jsp`	Controles de paginación y selector de registros por página, usados en todas las tablas.

Módulos de cuentas y perfil

Archivo	Función
`perfil/perfil.jsp`	Mi perfil: foto, nombre y tipo de documento editables; documento y correo bloqueados; botón Guardar.
`usuarios/usuarios.jsp`	Tabla de usuarios con búsqueda, filtros y acciones (administrador).
`usuarios/usuario_form.jsp`	Formulario para registrar o editar docentes, coordinadores y administradores, incluidos los datos bloqueados.
`usuarios/cuentas_pendientes.jsp`	Solicitudes de cuenta de estudiantes: aprobar (asignando el nivel) o rechazar.

Módulos de ideas y propuestas

Archivo	Función
`banco_ideas/banco_ideas.jsp`	Listado de ideas de docentes aprobadas; el estudiante puede elegir una.
`banco_ideas/idea_form.jsp`	Formulario para que el docente proponga una idea (título, descripción, objetivo y alcance).
`propuestas/propuestas_banco.jsp`	CRUD tipo tabla con código, descripción, estado y acciones de visualizar y gestionar, con filtro y paginación.
`propuestas/propuesta_form.jsp`	Formulario de propuesta propia (título, objetivo y alcance) y búsqueda del segundo estudiante por documento.
`propuestas/propuesta_detalle.jsp`	Detalle de la solicitud (icono de visualizar).
`propuestas/propuesta_decision.jsp`	Aprobar, aplazar o rechazar con observación (icono de gestionar; solo coordinador).

Módulos de proyectos y fases

Archivo	Función
`proyectos/asignacion.jsp`	El coordinador asigna director y evaluador (validando que sean distintos).
`proyectos/buscar_estudiante.jsp`	Consulta del proyecto de un estudiante por código o nombre.
`proyectos/seguimiento.jsp`	Seguimiento del proyecto con las seis tarjetas de fase (verdes y grises).
`proyectos/detalles_proyecto.jsp`	Ventana con los documentos de la fase (informe, rejilla, Turnitin, propuesta), su código, estado y enlace.
`proyectos/entrega_form.jsp`	Formulario para entregar un documento por enlace (solo la URL).
`docente_director/docente_director.jsp`	Tabla de proyectos del docente como director, con tarjetas de fase y acciones.
`docente_evaluador/docente_evaluador.jsp`	Tabla de proyectos del docente como evaluador, con el mismo modelo.

Otros módulos

Archivo	Función
`pqrs/pqrs.jsp`	Listado de PQRS con filtro y acciones.
`pqrs/pqrs_form.jsp`	Radicar una PQRS (prórroga con soporte de pago, cambio de director, petición, queja, reclamo o sugerencia).
`pqrs/pqrs_detalle.jsp`	Detalle de la PQRS y respuesta del comité.
`historial/historial.jsp`	Línea de tiempo de versiones de documentos y cambios de estado de un proyecto.
`informes/informe_coordinador.jsp`	Informe de estado por estudiante, director y evaluador.
`informes/informe_estudiante.jsp`	Informe personal: estado del anteproyecto y avance de las seis fases.
`base_documental/base_documental.jsp`	Seis tarjetas con enlaces externos (calendario, formatos, presentaciones y reglamentos).
`base_documental/enlaces_admin.jsp`	El administrador crea, edita, ordena o desactiva los enlaces.
`notificaciones/notificaciones.jsp`	Listado de avisos internos con opción de marcar como leídos.

Páginas de proceso (webapp/proceso)

Archivo	Función
`login_proc.jsp`	Valida credenciales y estado de la cuenta, crea la sesión y, si se marcó, el token de sesión persistente.
`registro_proc.jsp`	Valida el dominio del correo y que documento y correo no existan; crea la cuenta pendiente.
`perfil_proc.jsp`	Guarda nombre, tipo de documento y foto.
`usuario_proc.jsp`	Crea, edita o inactiva usuarios (administrador).
`cuenta_decision_proc.jsp`	Aprueba o rechaza una cuenta y asigna el nivel; registra el evento en el historial.
`idea_proc.jsp`	Crea ideas de docentes y registra la decisión del coordinador.

Archivo	Función
`propuesta_proc.jsp`	Crea propuestas, agrega al segundo estudiante y registra decisiones; al aprobar crea el proyecto con sus seis fases.
`asignacion_proc.jsp`	Guarda director y evaluador y notifica a los docentes.
`entrega_proc.jsp`	Registra una nueva versión de documento y marca la anterior como no vigente.
`revision_proc.jsp`	Registra la decisión del revisor, avanza o devuelve la fase y genera notificaciones e historial.
`pqrs_proc.jsp`	Radica y resuelve PQRS; aplica prórrogas y cambios de director.
`enlace_proc.jsp`	Guarda los cambios de los enlaces de la base documental.
`notificacion_proc.jsp`	Marca avisos como leídos.

Recursos estáticos y raíz del repositorio

Archivo	Función
`assets/css/estilos.css`	Estilos de la aplicación: colores, tarjetas, tablas, botones y diseño adaptable.
`assets/js/validaciones.js`	Validaciones en el navegador (campos obligatorios, correo, URL).
`assets/js/tablas.js`	Comportamiento de las tablas (búsqueda y selector de registros).
`assets/img/`	Logo, iconos y foto de perfil por defecto.
`README.md`	Descripción del proyecto, requisitos, instalación de Tomcat y MySQL, y cómo ejecutar el script SQL.
`.gitignore`	Archivos que no se suben a GitHub (credenciales de la conexión y archivos temporales).

9.4 Flujo típico de una página

- **Vista** (por ejemplo, *propuestas_banco.jsp*): incluye *conexion.jsp* y *seguridad.jsp*, consulta los datos con <sql:query> y los muestra con <c:forEach>, junto con *cabecera.jsp*, *menu.jsp* y *pie.jsp*.
- **Formulario:** el usuario completa los datos y los envía, junto con el token anti-CSRF, a una página de *proceso*.
- **Proceso** (por ejemplo, *propuesta_proc.jsp*): verifica la seguridad, valida los datos, ejecuta las sentencias SQL con <sql:param> dentro de una transacción y registra el evento en el historial.
- **Respuesta:** redirige con <c:redirect> a la vista y muestra un mensaje de éxito o error.

Anexo A. Script SQL de la base de datos (MySQL 8)

Script completo de creación de tablas, restricciones y datos iniciales. También se entrega como archivo independiente en el repositorio (carpeta `sql/`).

```
-- Sistema de Gestión de Proyectos de Grado - MySQL 8.x
SET NAMES utf8mb4;
CREATE DATABASE IF NOT EXISTS gestion_grado CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE gestion_grado;

CREATE TABLE rol (
  id_rol TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(30) NOT NULL,
  descripcion VARCHAR(120) NULL,
  PRIMARY KEY (id_rol),
  UNIQUE KEY uq_rol_nombre (nombre)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE usuario (
  id_usuario INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_rol TINYINT UNSIGNED NOT NULL,
  tipo_documento ENUM('CC','TI','CE','PASAPORTE','OTRO') NOT NULL DEFAULT 'CC',
  numero_documento VARCHAR(20) NOT NULL,
  nombre_completo VARCHAR(120) NOT NULL,
  correo VARCHAR(120) NOT NULL,
  contrasena_hash CHAR(64) NOT NULL,
  sal CHAR(32) NOT NULL,
  foto_url VARCHAR(300) NULL,
  nivel ENUM('TECNOLOGICO','PROFESIONAL') NULL,
  estado_cuenta ENUM('PENDIENTE','APROBADA','RECHAZADA') NOT NULL DEFAULT 'PENDIENTE',
  activo TINYINT(1) NOT NULL DEFAULT 1,
  id_aprobador INT UNSIGNED NULL,
  fecha_decision DATETIME NULL,
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_usuario),
  UNIQUE KEY uq_usuario_numero_documento (numero_documento),
  UNIQUE KEY uq_usuario_correo (correo),
  CONSTRAINT ck_usuario_nivel CHECK (estado_cuenta <> 'RECHAZADA' OR nivel IS NULL),
  CONSTRAINT fk_usuario_id_rol
    FOREIGN KEY (id_rol) REFERENCES rol(id_rol),
  CONSTRAINT fk_usuario_id_aprobador
    FOREIGN KEY (id_aprobador) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE sesion_persistente (
  id_sesion INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario INT UNSIGNED NOT NULL,
  token_hash CHAR(64) NOT NULL,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_expira DATETIME NOT NULL,
  PRIMARY KEY (id_sesion),
  UNIQUE KEY uq_sesion_persistente_token_hash (token_hash),
  CONSTRAINT fk_sesion_persistente_id_usuario
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE idea_banco (
  id_idea INT UNSIGNED NOT NULL AUTO_INCREMENT,
  codigo VARCHAR(15) NOT NULL,
  id_docente INT UNSIGNED NOT NULL,
  titulo VARCHAR(200) NOT NULL,
  descripcion TEXT NOT NULL,
  objetivo TEXT NULL,
  alcance TEXT NULL,
  estado_decision ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO') NOT NULL DEFAULT 'PENDIENTE',
  observacion TEXT NULL,
  id_revisor INT UNSIGNED NULL,
  disponible TINYINT(1) NOT NULL DEFAULT 1,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_decision DATETIME NULL,
  PRIMARY KEY (id_idea),
  UNIQUE KEY uq_idea_banco_codigo (codigo),
  CONSTRAINT fk_idea_banco_id_docente
    FOREIGN KEY (id_docente) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_idea_banco_id_revisor
```

```
    FOREIGN KEY (id_revisor) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE propuesta (
  id_propuesta INT UNSIGNED NOT NULL AUTO_INCREMENT,
  codigo VARCHAR(15) NOT NULL,
  id_idea INT UNSIGNED NULL,
  origen ENUM('BANCO','PROPIA') NOT NULL,
  titulo VARCHAR(200) NOT NULL,
  descripcion TEXT NOT NULL,
  objetivo TEXT NOT NULL,
  alcance TEXT NOT NULL,
  estado_decision ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO') NOT NULL DEFAULT 'PENDIENTE',
  observacion TEXT NULL,
  id_revisor INT UNSIGNED NULL,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_decision DATETIME NULL,
  PRIMARY KEY (id_propuesta),
  UNIQUE KEY uq_propuesta_codigo (codigo),
  UNIQUE KEY uq_propuesta_id_idea (id_idea),
  CONSTRAINT fk_propuesta_id_idea
    FOREIGN KEY (id_idea) REFERENCES idea_banco(id_idea),
  CONSTRAINT fk_propuesta_id_revisor
    FOREIGN KEY (id_revisor) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE propuesta_estudiante (
  id_propuesta INT UNSIGNED NOT NULL,
  id_estudiante INT UNSIGNED NOT NULL,
  es_responsable TINYINT(1) NOT NULL DEFAULT 0,
  PRIMARY KEY (id_propuesta, id_estudiante),
  CONSTRAINT fk_propuesta_estudiante_id_propuesta
    FOREIGN KEY (id_propuesta) REFERENCES propuesta(id_propuesta) ON DELETE CASCADE,
  CONSTRAINT fk_propuesta_estudiante_id_estudiante
    FOREIGN KEY (id_estudiante) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE fase (
  id_fase TINYINT UNSIGNED NOT NULL,
  numero TINYINT UNSIGNED NOT NULL,
  nombre VARCHAR(40) NOT NULL,
  descripcion VARCHAR(200) NULL,
  PRIMARY KEY (id_fase),
  UNIQUE KEY uq_fase_numero (numero)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE proyecto (
  id_proyecto INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_propuesta INT UNSIGNED NOT NULL,
  id_director INT UNSIGNED NULL,
  id_evaluador INT UNSIGNED NULL,
  fase_actual TINYINT UNSIGNED NOT NULL DEFAULT 1,
  estado_general ENUM('EN_CURSO','APTO_GRADUACION','CERRADO') NOT NULL DEFAULT 'EN_CURSO',
  fecha_inicio DATE NOT NULL,
  PRIMARY KEY (id_proyecto),
  UNIQUE KEY uq_proyecto_id_propuesta (id_propuesta),
  CONSTRAINT ck_proyecto_dir_eval
    CHECK (id_director IS NULL OR id_evaluador IS NULL OR id_director <> id_evaluador),
  CONSTRAINT fk_proyecto_id_propuesta
    FOREIGN KEY (id_propuesta) REFERENCES propuesta(id_propuesta),
  CONSTRAINT fk_proyecto_id_director
    FOREIGN KEY (id_director) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_proyecto_id_evaluador
    FOREIGN KEY (id_evaluador) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_proyecto_fase_actual
    FOREIGN KEY (fase_actual) REFERENCES fase(id_fase)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE proyecto_fase (
  id_proyecto_fase INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_proyecto INT UNSIGNED NOT NULL,
  id_fase TINYINT UNSIGNED NOT NULL,
  estado ENUM('PENDIENTE','EN_CURSO','APROBADA','APLAZADA','RECHAZADA') NOT NULL DEFAULT 'PENDIENTE',
  fecha_inicio DATETIME NULL,
  fecha_fin DATETIME NULL,
  PRIMARY KEY (id_proyecto_fase),
  UNIQUE KEY uq_proyecto_fase (id_proyecto, id_fase),
  CONSTRAINT fk_proyecto_fase_id_proyecto
```

```
    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto),
  CONSTRAINT fk_proyecto_fase_id_fase
    FOREIGN KEY (id_fase) REFERENCES fase(id_fase)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE documento (
  id_documento INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_proyecto_fase INT UNSIGNED NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  version_num SMALLINT UNSIGNED NOT NULL DEFAULT 1,
  enlace_url VARCHAR(500) NOT NULL,
  id_subido_por INT UNSIGNED NOT NULL,
  vigente TINYINT(1) NOT NULL DEFAULT 1,
  comentario VARCHAR(300) NULL,
  fecha_subida DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_documento),
  UNIQUE KEY uq_doc_version (id_proyecto_fase, nombre, version_num),
  CONSTRAINT fk_documento_id_proyecto_fase
    FOREIGN KEY (id_proyecto_fase) REFERENCES proyecto_fase(id_proyecto_fase),
  CONSTRAINT fk_documento_id_subido_por
    FOREIGN KEY (id_subido_por) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE revision (
  id_revision INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_proyecto_fase INT UNSIGNED NOT NULL,
  id_documento INT UNSIGNED NULL,
  id_revisor INT UNSIGNED NOT NULL,
  rol_revisor ENUM('DIRECTOR','EVALUADOR','COMITE') NOT NULL,
  decision ENUM('APROBADO','APLAZADO','RECHAZADO') NOT NULL,
  observaciones TEXT NULL,
  fecha_revision DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_revision),
  CONSTRAINT fk_revision_id_proyecto_fase
    FOREIGN KEY (id_proyecto_fase) REFERENCES proyecto_fase(id_proyecto_fase),
  CONSTRAINT fk_revision_id_documento
    FOREIGN KEY (id_documento) REFERENCES documento(id_documento),
  CONSTRAINT fk_revision_id_revisor
    FOREIGN KEY (id_revisor) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE pqrs (
  id_pqrs INT UNSIGNED NOT NULL AUTO_INCREMENT,
  codigo VARCHAR(15) NOT NULL,
  id_proyecto INT UNSIGNED NULL,
  id_solicitante INT UNSIGNED NOT NULL,
  tipo ENUM('PRORROGA','CAMBIO_DIRECTOR','PETICION','QUEJA','RECLAMO','SUGERENCIA') NOT NULL,
  asunto VARCHAR(150) NOT NULL,
  descripcion TEXT NOT NULL,
  enlace_soporte VARCHAR(500) NULL,
  id_nuevo_director INT UNSIGNED NULL,
  estado ENUM('PENDIENTE','APROBADO','APLAZADO','RECHAZADO') NOT NULL DEFAULT 'PENDIENTE',
  respuesta TEXT NULL,
  id_resuelto_por INT UNSIGNED NULL,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_resolucion DATETIME NULL,
  PRIMARY KEY (id_pqrs),
  UNIQUE KEY uq_pqrs_codigo (codigo),
  CONSTRAINT fk_pqrs_id_proyecto
    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto),
  CONSTRAINT fk_pqrs_id_solicitante
    FOREIGN KEY (id_solicitante) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_pqrs_id_nuevo_director
    FOREIGN KEY (id_nuevo_director) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_pqrs_id_resuelto_por
    FOREIGN KEY (id_resuelto_por) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE notificacion (
  id_notificacion BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario INT UNSIGNED NOT NULL,
  id_proyecto INT UNSIGNED NULL,
  tipo VARCHAR(30) NOT NULL,
  mensaje VARCHAR(255) NOT NULL,
  leida TINYINT(1) NOT NULL DEFAULT 0,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_notificacion),
  CONSTRAINT fk_notificacion_id_usuario
```

```
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE,
  CONSTRAINT fk_notificacion_id_proyecto
    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE historial_version (
  id_historial BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario INT UNSIGNED NOT NULL,
  id_proyecto INT UNSIGNED NULL,
  id_documento INT UNSIGNED NULL,
  entidad ENUM('USUARIO','IDEA','PROPUESTA','PROYECTO','FASE','DOCUMENTO','PQRS') NOT NULL,
  id_entidad INT UNSIGNED NOT NULL,
  accion VARCHAR(40) NOT NULL,
  estado_anterior VARCHAR(20) NULL,
  estado_nuevo VARCHAR(20) NULL,
  version_num SMALLINT UNSIGNED NULL,
  detalle VARCHAR(500) NULL,
  fecha_evento DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_historial),
  KEY ix_hist_proyecto (id_proyecto, fecha_evento),
  CONSTRAINT fk_historial_version_id_usuario
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_historial_version_id_proyecto
    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto),
  CONSTRAINT fk_historial_version_id_documento
    FOREIGN KEY (id_documento) REFERENCES documento(id_documento)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE enlace_documental (
  id_enlace TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(80) NOT NULL,
  descripcion VARCHAR(200) NULL,
  url VARCHAR(500) NOT NULL,
  icono VARCHAR(30) NULL,
  orden TINYINT UNSIGNED NOT NULL DEFAULT 0,
  activo TINYINT(1) NOT NULL DEFAULT 1,
  PRIMARY KEY (id_enlace)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Datos iniciales
INSERT INTO rol (nombre, descripcion) VALUES
  ('ADMINISTRADOR','Comité: gestiona usuarios'),
  ('COORDINADOR','Comité: aprueba y asigna'),
  ('ESTUDIANTE','Desarrolla el trabajo de grado'),
  ('DOCENTE','Puede ser director y evaluador');

INSERT INTO fase (id_fase, numero, nombre, descripcion) VALUES
  (1,1,'Pago','El estudiante sube el comprobante; se notifica al comité'),
  (2,2,'Propuesta I','El estudiante envía el anteproyecto al director'),
  (3,3,'Propuesta II','El director envía la propuesta al evaluador y al comité'),
  (4,4,'Informe I','El estudiante envía el informe al director'),
  (5,5,'Informe II','El director envía el informe al evaluador y al comité'),
  (6,6,'Revisión final','Evaluador y comité aprueban o aplazan');

INSERT INTO enlace_documental (nombre, descripcion, url, orden) VALUES
  ('Calendario académico','Fechas del semestre','#',1),
  ('Formatos institucionales','Formatos de grado','#',2),
  ('Presentación de propuesta','Guía para la propuesta','#',3),
  ('Presentación de informe final','Guía del informe final','#',4),
  ('Reglamento de estímulos','Estímulos para trabajos de grado','#',5),
  ('Reglamento de trabajos de grado','Reglamento vigente','#',6);

-- Administrador inicial (cambiar la contraseña tras el primer ingreso)
INSERT INTO usuario (id_rol, tipo_documento, numero_documento, nombre_completo,
                     correo, contrasena_hash, sal, estado_cuenta)
VALUES (1,'CC','0000000000','Administrador del sistema','admin@correo.uts.edu.co',
        SHA2(CONCAT('3f9a1c7e5b2d4a60','CambiarClave*1'),256),'3f9a1c7e5b2d4a60','APROBADA');
```